CSE1022 – Fundamentals of Data Structure and Algorithms

Module 2 – Arrays and Linear Data Structures

Dr. Vignesh Ramamoorthy. H

Associate Professor

AIIT – AUB

AMITY UNIVERSITY BENGALURU

Module 2 – Arrays and Linear Data Structures

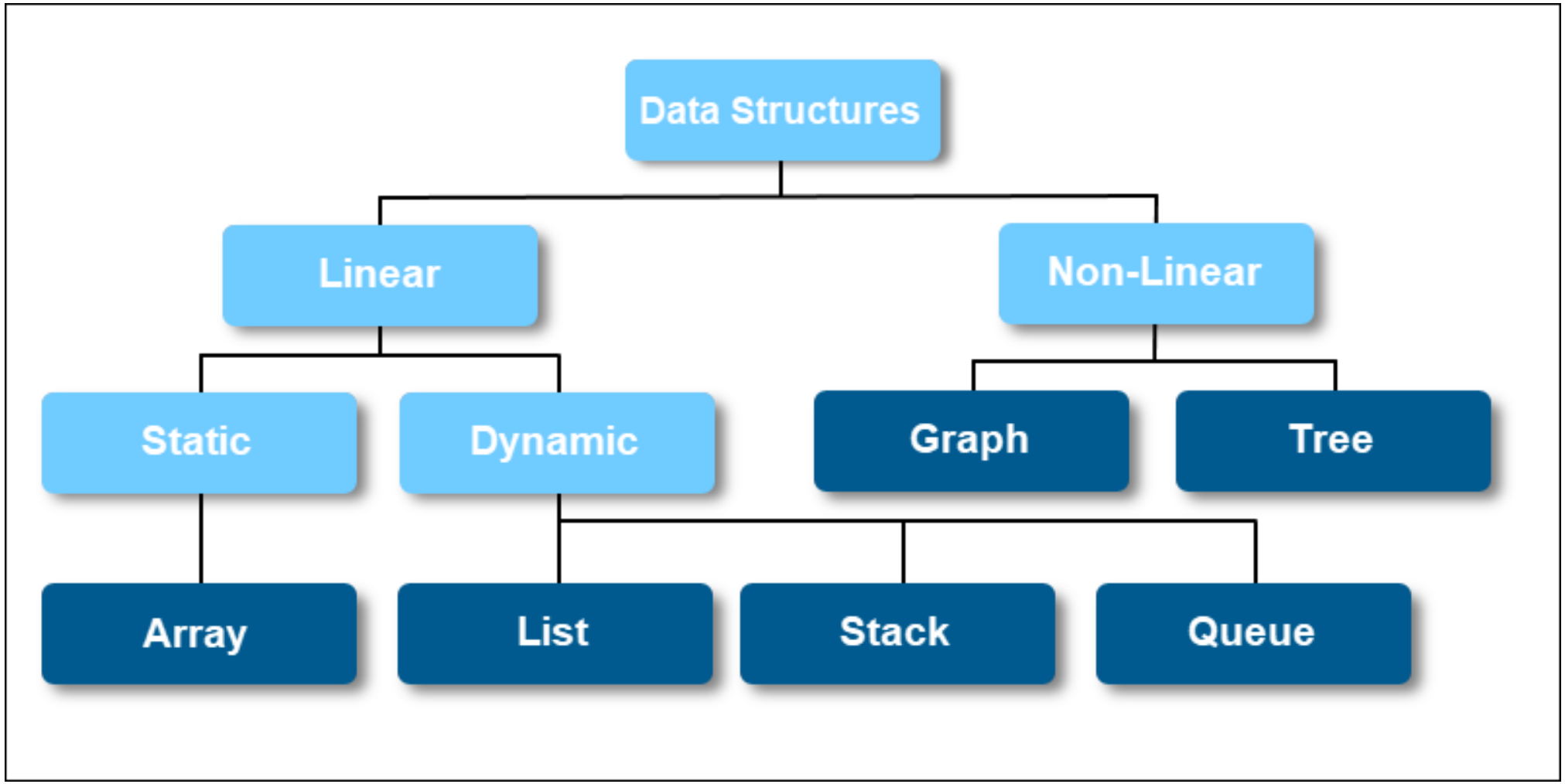
- Classification: Linear vs Non-linear Data Structures
- Arrays: Representation, Traversal, Insertion and Deletion
- Searching in Arrays
- Applications of Arrays

AMITY UNIVERSITY BENGALURU

Module 2 – Arrays and Linear Data Structures

- **Classification: Linear vs Non-linear Data Structures**
- Arrays: Representation, Traversal, Insertion and Deletion
- Searching in Arrays
- Applications of Arrays

Classification: Linear and Non-Linear Data Structures



Classification : Linear and Non-Linear Data Structures

A **linear data structure** is one in which elements are arranged sequentially, one after another.

- Each element has a **unique predecessor and successor** (except the first and last).
- Traversal is done in a single run (one after another).

Examples:

- ✓ **Array** → elements stored in contiguous memory
- ✓ **Linked List** → elements linked one after another
- ✓ **Stack** → LIFO (Last In, First Out)
- ✓ **Queue** → FIFO (First In, First Out)

Linear Data Structures

- ✓ Data elements are **arranged in a straight line**.
- ✓ Only **one predecessor and one successor** (except the first and last).
- ✓ Traversal happens in **one level** (you go step by step).
- ✓ Suitable for problems where data needs to be **processed in order**.

Key Characteristics:

- ✓ Easy to implement.
- ✓ Memory use is **continuous** (in arrays) or **sequentially linked** (in linked lists).
- ✓ Operations like insertion, deletion, and traversal are straightforward but sometimes **less efficient** (e.g., inserting in the middle of an array requires shifting).

Linear Data Structures

Examples with Uses:

- ✓ **Array** → Used in lookup tables, image processing, matrices.
- ✓ **Linked List** → Dynamic memory allocation, music playlists.
- ✓ **Stack** → Undo/redo operations in editors, browser back button.
- ✓ **Queue** → Printer job scheduling, CPU task scheduling.

Non Linear Data Structures

- ✓ Data elements are **not arranged sequentially**.
- ✓ They are stored in a **hierarchical (tree)** or **network (graph)** structure.
- ✓ A node can connect to **multiple nodes**.
- ✓ Traversal can happen through **multiple paths**.

Key Characteristics:

- ✓ Memory is **scattered** (not continuous).
- ✓ Complex relationships are supported (e.g., parent-child in trees, friends in social networks).
- ✓ More flexible but harder to implement.

Non Linear Data Structures

Examples with Uses:

- ✓ **Tree** → Used in file systems, XML/HTML DOM, decision-making systems.
- ✓ **Binary Search Tree (BST)** → Efficient searching & sorting.
- ✓ **Heap** → Priority queues, scheduling.
- ✓ **Graph** → Google Maps, Facebook friend networks, airline route maps.

Comparison: Linear Data Structure Vs Non-Linear Data Structure

Aspect	*Linear Data Structure*	*Non-Linear Data Structure*
Arrangement	Sequential, one after another	Hierarchical or network-based
Traversal	Single level (start → end)	Multiple levels/paths
Predecessor/Successor	Each element has one (except first/last)	An element can have many relationships
Complexity	Simple to implement	More complex (uses pointers/links)

Comparison: Linear Data Structure Vs Non-Linear Data Structure

Aspect	*Linear Data Structure*	*Non-Linear Data Structure*
Memory Utilization	Continuous (arrays) or sequential	Scattered, depends on references
Examples	Array, Linked List, Stack, Queue	Tree, Graph, Heap
Applications	Task scheduling, simple storage, undo/redo, queues in OS	File systems, social networks, AI, compilers, shortest path algorithms

Comparison: Linear Data Structure Vs Non-Linear Data Structure

Summary:

- Linear DS → Best for ordered data (arrays, stacks, queues).
- Non-Linear DS → Best for complex relationships (trees, graphs).

AMITY UNIVERSITY BENGALURU

Module 2 – Arrays and Linear Data Structures

- Classification: Linear vs Non-linear Data Structures
- **Arrays: Representation, Traversal, Insertion and Deletion**
- Searching in Arrays
- Applications of Arrays

Linear Array

An **array** is a linear data structure used to store **multiple elements of the same data type** in **contiguous memory locations**.

Each element in an array is accessed using an **index (position)**.

- Index usually starts from **0**.
- Elements are stored sequentially in memory.
- Accessing elements is very fast using the index.

Linear Array

Example

Given Array

A = [10, 20, 30, 40, 50]

Index	0	1	2	3	4
Value	10	20	30	40	50

Here:

- `A[0] = 10`
- `A[2] = 30`
- `A[4] = 50`

Key Points

- Stores **same type of data**
- Elements stored **sequentially**
- **Fast access** using **index**

Array

Array Representation

- An **array** is a linear data structure used to store **multiple values of the same data type** in a **single variable**.
- The elements in an array are stored in **continuous memory locations.**
- Each element is accessed using an **index (position number)**.

Formula:

$$\text{Address}(A[i]) = \text{Base_Address} + (i \times \text{Size_of_Element})$$

where:

- **Base_Address** = address of first element
- **i** = index of element
- **Size_of_Element** = size of each element in bytes

Array

Formula: `Address(A[i]) = Base_Address + (i × Size_of_Element)`

Array Representation

Example

Assume:

- Base address = 1000
- Size of integer = 4 bytes

Find address of `A[3]`

```
Address(A[3]) = 1000 + (3 × 4)
              = 1000 + 12
              = 1012
```

So, the element `A[3]` is stored at memory location **1012**.

Array

Array Representation

Algorithm

```
Step 1: Start
Step 2: Input Base_Address, index i, and Size_of_Element
Step 3: Calculate Address = Base_Address +
                            (i × Size_of_Element)
Step 4: Display Address
Step 5: Stop
```

Let us know!

Array Representation

Standard Sizes of Data Types

The size of elements depends on the **programming language** and **system (compiler/architecture)**, but commonly:

Data Type	*Typical Size*
char	1 byte
int	4 bytes
float	4 bytes
double	8 bytes

How to Find the Size (programmatically)

```
sizeof(char)
sizeof(int)
sizeof(float)
sizeof(double)
```

Array

Array Representation

Algorithm Explanation

- First, the **base address of the array** is given.
- The **index of the element** whose address we want is taken as input.
- The **size of each element** (for example, 4 bytes for integers) is known.
- Using the formula, the address is calculated.
- The calculated address tells **where the element is stored in memory**.

This is how the computer **quickly finds any element in an array using its index**.

Problem 1: Find the Memory Address of an Array Element

- An integer array **A[ ]** has a **base address of 5000**.
- Each element occupies **4 bytes** of memory.
- Find the **memory address of A[6]**.

Formula

Address(A[i]) = Base Address + (i × Size of element)

Solution

Given:

Base Address = 5000

Size of element = 4 bytes

Index i = 6

Problem 1: Find the Memory Address of an Array Element

Calculation:

```
Address(A[6]) = 5000 + (6 × 4)
              = 5000 + 24
              = 5024
```

Answer

The **memory address of A[6] is 5024**.

Problem 2: Find the Position of an Element Using Address

- An array **B[ ]** has a **base address of 2000**.
- Each element occupies **2 bytes**.
- The **address of an element is 2014**.
- Find the **index of that element**.

Formula

Address(A[i]) = Base Address + (i × Size of the element)

Rearranging the formula:

```
i = (Address - Base Address) / Size
```

Array Representation

Let us know!

Given Formula: $\text{Address}(A[i]) = \textit{Base Address} + (i \times \textit{Size})$

We want to **find** i(index), so we rearrange the formula.

Start with:

$$\text{Address} = \textit{Base Address} + (i \times \textit{Size})$$

Step 1: Subtract Base Address from both sides

$$\text{Address} - \textit{Base Address} = i \times \textit{Size}$$

Move **Base Address** to the left side.

Step 2: Divide both sides by Size

$$\frac{\text{Address} - \textit{Base Address}}{\textit{Size}} = i$$

Final Formula

$$i = \frac{\textit{Address} - \textit{Base Address}}{\textit{Size}}$$

Problem 2: Find the Position of an Element Using Address

Solution

Given:

Base Address = 2000

Address = 2014

Size = 2 bytes

Calculation:

```
i = (2014 – 2000) / 2
i = 14 / 2
i = 7
```

Answer

The element is located at **index 7**, i.e., **B[7]**.

Array

Array Traversal

Traversal means **visiting each element of the array one by one** to perform some operation like **printing or calculating values**.

Example

Array

```
A = [5, 10, 15, 20]
```

Traversal output

```
5 10 15 20
```

Array

Array Traversal

Algorithm

```
Step 1: Start
Step 2: Take array A and its size n
Step 3: Set i = 0
Step 4: Repeat while i < n
            Print A[i]
            Increase i by 1
Step 5: Stop
```

Array

Array Traversal

Algorithm Explanation

- First, we start the program.
- The array and its size are given as input.
- A variable **i** is used as a counter and starts from **0**.
- The loop runs until **i becomes equal to n** (size of array).
- In each step, the element **A[i]** is printed.
- After printing, **i is increased by 1**.
- This continues until all elements are visited.

So, every element in the array is displayed once.

Array

Insertion in Array

Insertion means **adding a new element into the array at a specific position**.
Since array elements are stored continuously, we must **shift elements to the right** to create space.

Example
Original array
```
A = [10, 20, 30, 40]
```

Insert **25 at position 2**

Array

Insertion in Array

Example

Original array

```
A = [10, 20, 30, 40]
```

Insert **25 at position 2**

Step 1: Shift elements

```
10 20 30 40
10 20 _ 30 40
```

Step 2: Insert value

```
10 20 25 30 40
```

Array

Insertion in Array

Algorithm

```
Step 1: Start
Step 2: Take array A and size n
Step 3: Enter position pos and new element x
Step 4: Move elements one step to the right
starting from the end
Step 5: Place x at position pos
Step 6: Increase array size by 1
Step 7: Stop
```

Array

Insertion in Array

Algorithm Explanation

- First, the array and its current size are given.
- The user enters the **position** where the element must be inserted.
- The user also enters the **new element**.
- To create space, elements are **shifted one position to the right**.
- After shifting, the new element is placed in the required position.
- The size of the array is increased by **one**.

This successfully inserts the new element.

Array

Insertion in Array – Exercise

Given

Array $A = [10, 20, 30, 40, 50, 60]$

Size $n = 6$

Insert **x = 25** at **position pos = 2** (0-based index)

Steps:-

Step 1: Start

Step 2: Array $A = [10, 20, 30, 40, 50, 60]$,$n = 6$

Step 3:

Position $pos = 2$, New element $x = 25$

Insertion in Array – Exercise

Step 4: Move elements to the right

Shift elements from end to position:

- **Move A[5] → A[6] →** `[10, 20, 30, 40, 50, 60, 60]`
- **Move A[4] → A[5] →** `[10, 20, 30, 40, 50, 50, 60]`
- **Move A[3] → A[4] →** `[10, 20, 30, 40, 40, 50, 60]`
- **Move A[2] → A[3] →** `[10, 20, 30, 30, 40, 50, 60]`

Insertion in Array – Exercise

Step 5: Place x at position pos

Insert 25 at index 2:

```
[10, 20, 25, 30, 40, 50, 60]
```

Step 6: Increase size

New size $n = 7$

Final Array

[10, 20, 25, 30, 40, 50, 60]

Array

Deletion in Array

- Deletion means **removing an element from a particular position in the array**.
- After deleting, the remaining elements must **shift to the left** to fill the gap.

Example

Original array

```
A = [10, 20, 30, 40, 50]
```

Delete element at **position 2**

Step 1: Remove 30

```
10 20 _ 40 50
```

Step 2: Shift elements

```
10 20 40 50
```

Array

Deletion in Array

Algorithm

```
Step 1: Start
Step 2: Take array A and size n
Step 3: Enter the position pos to delete
Step 4: Move all elements after pos one step to the left
Step 5: Reduce array size by 1
Step 6: Stop
```

Array

Deletion in Array

Algorithm Explanation

- First, the array and its size are given.
- The user specifies the **position of the element to delete**.
- The element at that position is removed.
- All elements **after that position move one step left**.
- This fills the empty space created by deletion.
- Finally, the array size is **reduced by one**.

Array

Deletion in Array – Exercise

Given

Array $A = [10, 20, 30, 40, 50, 60]$

Size $n = 6$

Delete element at **position pos = 2** (0-based index)

Steps:

Step 1: Start

Step 2: Array $A = [10, 20, 30, 40, 50, 60]$,$n = 6$

Step 3:

Position to delete $pos = 2$(element = 30)

Deletion in Array – Exercise

Step 4: Move elements to the left

Shift elements after position:

- **Move A[3] → A[2] →** `[10, 20, 40, 40, 50, 60]`
- **Move A[4] → A[3] →** `[10, 20, 40, 50, 50, 60]`
- **Move A[5] → A[4] →** `[10, 20, 40, 50, 60, 60]`

Deletion in Array – Exercise

Step 5: Reduce size

New size $n = 5$

Final Array

[10, 20, 40, 50, 60]

Array

Traversal	O(n)
Insertion	O(n)
Deletion	O(n)

Time Complexity of Array Operations

- **Traversal – O(n)**
  All elements in the array must be **visited one by one**, so the time depends on the number of elements (**n**).

- **Insertion – O(n)**
  When inserting an element (especially in the **beginning or middle**), existing elements must be **shifted to the right**, which may take **n steps**.

- **Deletion – O(n)**
  After deleting an element, the remaining elements may need to be **shifted to the left** to fill the empty space, which can take **up to n steps**.

Searching in Arrays

- Searching is the process of **finding whether a particular element exists in an array and identifying its position**.
- Two common searching methods used in arrays are

  (1) Linear Search

  (2) Binary Search

AMITY UNIVERSITY BENGALURU

Module 2 – Arrays and Linear Data Structures

- Classification: Linear vs Non-linear Data Structures
- Arrays: Representation, Traversal, Insertion and Deletion
- **Searching in Arrays**
- Applications of Arrays

Array

Searching in Arrays – Linear Search

- **Linear search** is the simplest method used to find an element in an array.
- The search **starts from the first element** of the array.
- Each element is **checked one by one** until the required element is found.
- The **search value is compared with every element** in the array.
- If a **match is found**, the search stops and the **position of the element is returned**.
- If **all elements are checked and no match is found**, the result shows that the **element is not present in the array**.
- Works with **both sorted and unsorted arrays**.
- Linear Search is also known as Sequential Search

Array

Searching in Arrays – Linear Search

Example

Consider the array:

```
[15, 25, 30, 45, 50]
```

If we search for **45**, the process will compare:

- 15 → not equal
- 25 → not equal
- 30 → not equal
- 45 → found

Thus, the element is found after checking several elements sequentially.

Searching in Arrays – Linear Search

Time Complexity

The worst case occurs when the element is **at the last position or not present**, so all elements must be checked.

Time Complexity: O(n)

Array

Searching in Arrays – Binary Search

- Binary search is a **method to find an element in a sorted array**.
- It first **checks the middle element of the array**.
- If the middle element is the **same as the value we are searching**, the element is found.
- If the value is **smaller than the middle element**, the search continues in the **left side** of the array.
- If the value is **greater than the middle element**, the search continues in the **right side** of the array.
- Each time, the **search area becomes half**, so the element can be found faster.
- Binary search **works only when the array is sorted**.

Array

Searching in Arrays – Binary Search

Example 1

Consider the sorted array:

```
[10, 20, 30, 40, 50]
```

If we search for **40**:

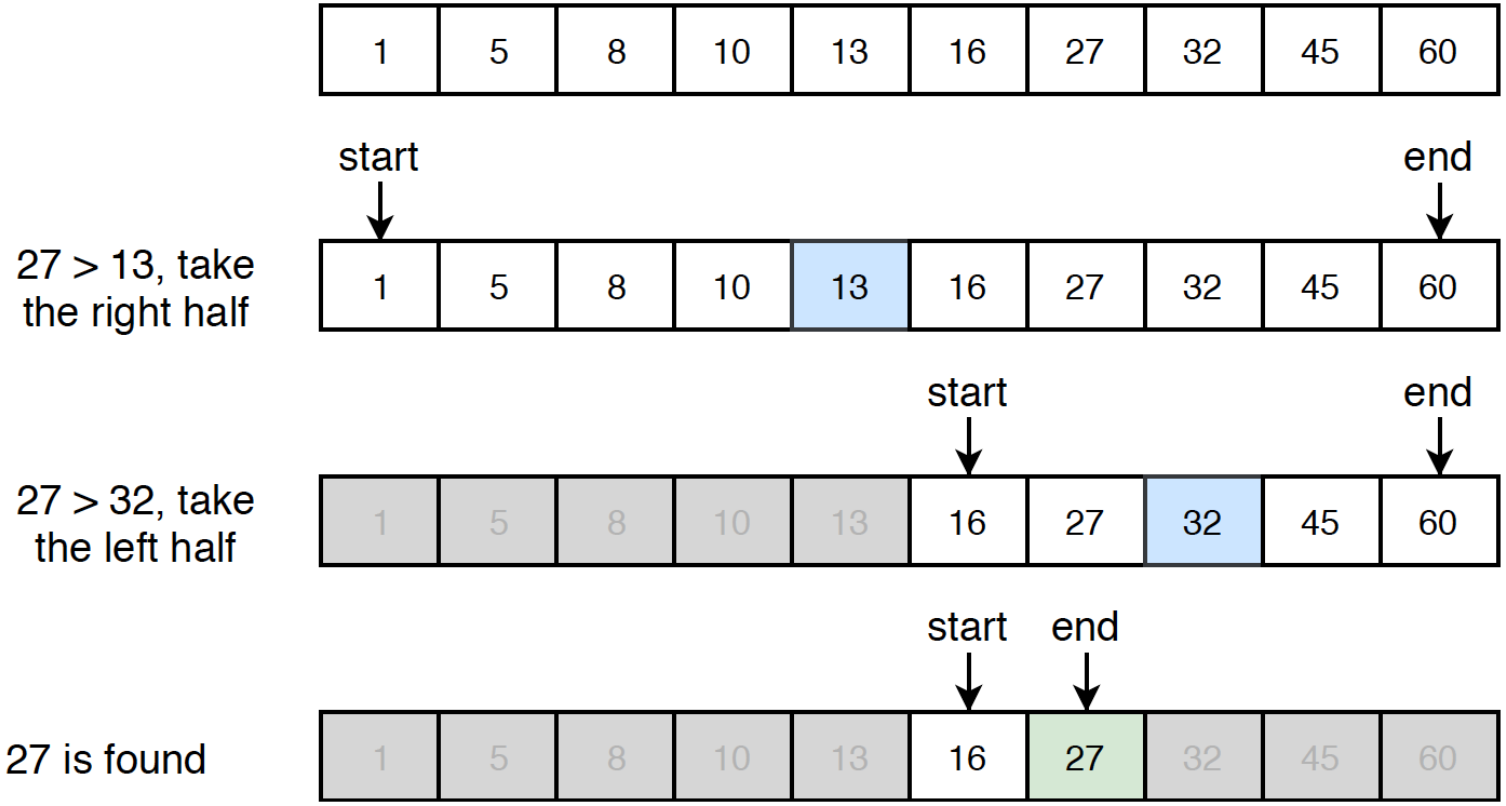
1. The middle element is **30**.
2. Since **40 is greater than 30**, the search moves to the **right half**.
3. The next middle element becomes **40**, so the element is found.

Instead of checking all elements, the search area becomes **smaller after each step**.

Searching in Arrays – Binary Search

Example 2

Search 27 in a sorted array with 10 elements



Array

Searching in Arrays – Binary Search

Example 3

	0	1	2	3	4	5	6
Search 50	11	17	18	45	50	71	95
	L=0	1	2	M=3	4	5	H=6
50 > 45 Take 2nd half	11	17	18	45	50	71	95
	0	1	2	3	L=4	M=5	M=6
50 < 71 Take 1st half	11	17	18	45	50	71	95
	0	1	2	3	L=4 M=4		
50 found at position 4	11	17	18	45	50	71	95
					done		

Searching in Arrays – Binary Search

Time Complexity

Since the array is divided into two parts at every step, the number of comparisons decreases quickly.

Time Complexity: O(log n)

Array

Time Complexity

Operation	*Time Complexity*	*Explanation*
Traversal	O(n)	Every element in the array must be visited once.
Insertion	O(n)	Elements may need to be **shifted to the right** to insert a new value.
Deletion	O(n)	Elements may need to be **shifted to the left** after deleting an element.
Linear Search	O(n)	The algorithm checks **each element one by one** until the element is found.
Binary Search	O(log n)	The array is **repeatedly divided into two halves**, so the search becomes faster.

AMITY UNIVERSITY BENGALURU

Module 2 – Arrays and Linear Data Structures

- Classification: Linear vs Non-linear Data Structures
- Arrays: Representation, Traversal, Insertion and Deletion
- Searching in Arrays
- **Applications of Arrays**

Applications of Arrays

- **Storing data**

  Store multiple values like marks, salaries, etc.

- **Matrices (2D arrays)**

  Used in matrix operations and image processing

- **Implementing data structures**

  Used to build stacks and queues

- **Searching and sorting**

  Used in algorithms like binary search and sorting methods

Applications of Arrays

- **Strings**

  Strings are stored as arrays of characters

- **Graphs**

  Used in adjacency matrix representation

- **Dynamic programming**

  Store intermediate results to solve problems efficiently

- **Lookup tables**

  Store precomputed values for quick access

Practice Exercises:

Array Insertion

1. LA = [10, 20, 30, 40], N = 4, K = 3, ITEM = 25
2. LA = [5, 15, 25, 35, 45], N = 5, K = 2, ITEM = 10
3. LA = [2, 4, 6, 8, 10], N = 5, K = 6, ITEM = 12
4. LA = [100, 200, 300], N = 3, K = 1, ITEM = 50
5. LA = [1, 3, 5, 7, 9], N = 5, K = 4, ITEM = 6

Practice Exercises:

Array Deletion

1. LA = [10, 20, 30, 40, 50], N = 5, K = 3
2. LA = [5, 10, 15, 20, 25], N = 5, K = 1
3. LA = [11, 22, 33, 44, 55, 66], N = 6, K = 6
4. LA = [2, 4, 6, 8, 10], N = 5, K = 2
5. LA = [9, 8, 7, 6, 5], N = 5, K = 5

Practice Exercises:

Array Searching

1. LA = [10, 20, 30, 40, 50], N = 5, ITEM = 30
2. LA = [5, 15, 25, 35, 45], N = 5, ITEM = 50
3. LA = [2, 4, 6, 8, 10], N = 5, ITEM = 6
4. LA = [100, 200, 300, 400], N = 4, ITEM = 500
5. LA = [1, 3, 5, 7, 9, 11], N = 6, ITEM = 7

THANK YOU

Dr. Vignesh Ramamoorthy. H

Associate Professor

AIIT – AUB